

CS 591 Programming Project 3: SGD Optimizer and Neural Network

Team Members:

- Sisir Khadka, [REDACTED], Undergraduate
- Aven Pacha Pinto, [REDACTED], Graduate
- MD Faisal Khan, [REDACTED], Graduate
- Anna Stahl, [REDACTED], Undergraduate
- Nidah Fariha, [REDACTED], Undergraduate

Abstract

In this project, we explore the foundational optimization techniques of machine learning alongside a practical deep learning application. In Part 1, we implement Gradient Descent (GD) and Stochastic Gradient Descent (SGD) from scratch to fit a polynomial curve to 100 data points. We conduct extensive experiments across varying feature counts and learning rates, visualizing the model's learning process over time. In Part 2, we design and train a deep learning Feed-Forward Neural Network using TensorFlow to solve a real-world computer vision challenge: clothing classification using the Fashion-MNIST dataset.

Part 1: Curve Fitting using Gradient Descent & Stochastic Gradient Descent

1. Problem Description & Methodology

The objective of this task is to find the optimal line of best fit for 100 provided coordinate points. To accomplish this, we utilize polynomial regression, treating the powers of X as distinct features.

To prevent mathematical overflow (exploding gradients) when raising our input variables to higher powers, we first normalized the X data to a scale of 0 to 1. This crucial data preprocessing step allowed us to safely test higher-order polynomials and maintain stable learning.

We implemented two optimization algorithms from scratch:

- Gradient Descent (GD): Calculates the gradient of the entire dataset to update the feature weights once per epoch.
- Stochastic Gradient Descent (SGD): Shuffles the dataset and updates the feature weights for every individual data point, resulting in faster but noisier convergence.

2. Implementation Details & Bonus Objectives

To thoroughly evaluate our optimizers, we expanded our scope to capture all available bonus points:

- Bonus 1 (Extended Features): In addition to the required 3-feature model ($Y = a * X^2 + b * X + c$), we successfully modeled a 4-feature cubic curve ($Y = a * X^3 + b * X^2 + c * X + d$) and a 5-feature quartic curve.
- Bonus 2 (Learning Rate Exploration): We experimented with the baseline learning rate of 0.01, as well as two additional rates: 0.05 and 0.10.
- Bonus 3 (Epoch Visualization): Our implementation successfully records the weight history. We generated visualizations that overlay the curve for every epoch, demonstrating exactly how the line shifts as the optimizer learns.

3. Results and Analysis

(Note: Below is a selection of our results highlighting key observations. Please see the attached source code for the full suite of generated experiments).

The Impact of Features (Degrees of Polynomial)

As we increased the number of features from 3 to 5, the model's ability to capture the complex, wave-like nature of the data points improved significantly.

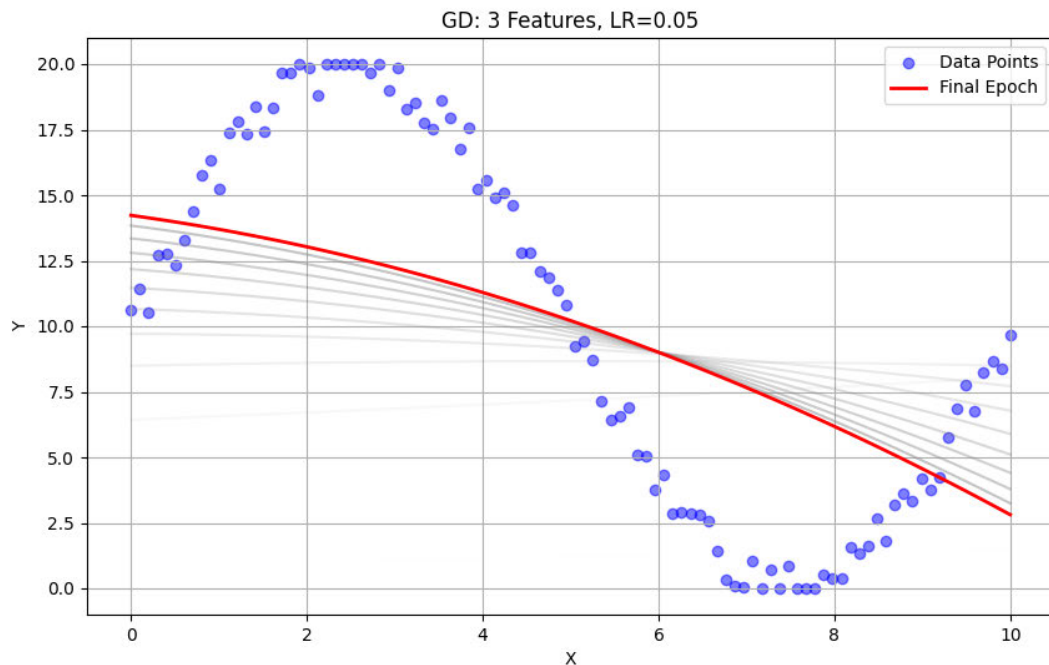


Figure 1: A 3-feature (quadratic) model provides a basic parabolic fit, missing the nuanced peaks and valleys.

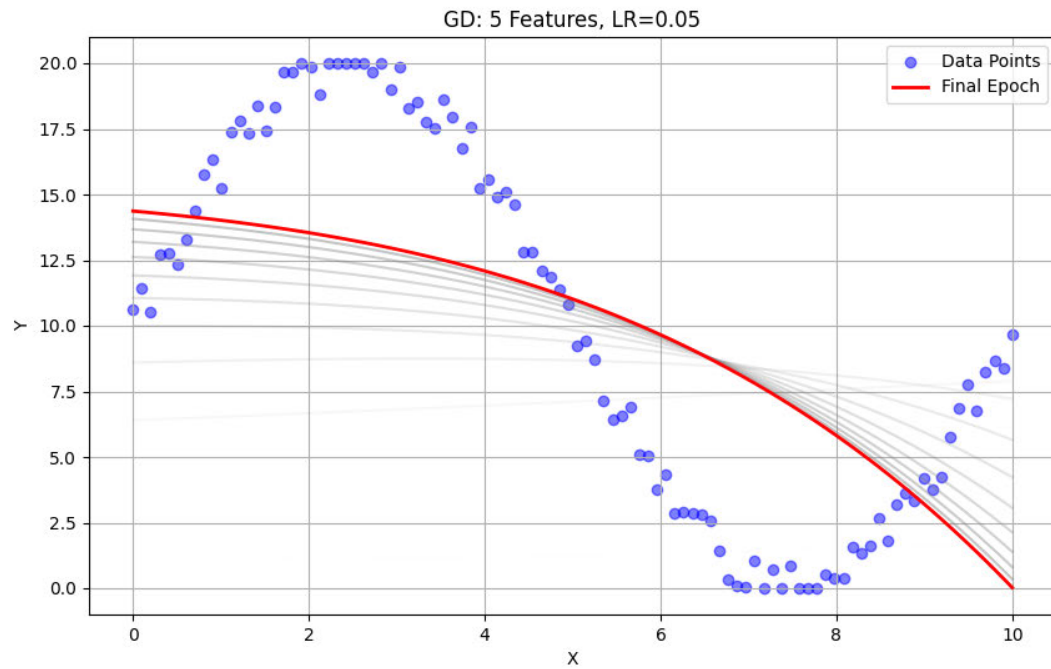


Figure 2: A 5-feature (quartic) model tightly contours to the true underlying distribution of the data.

Gradient Descent vs. Stochastic Gradient Descent

When comparing the two optimizers, GD provided smooth, calculated updates. In contrast, SGD reached an acceptable fit much faster due to the frequent weight updates, though its final epochs showed minor fluctuations around the global minimum.

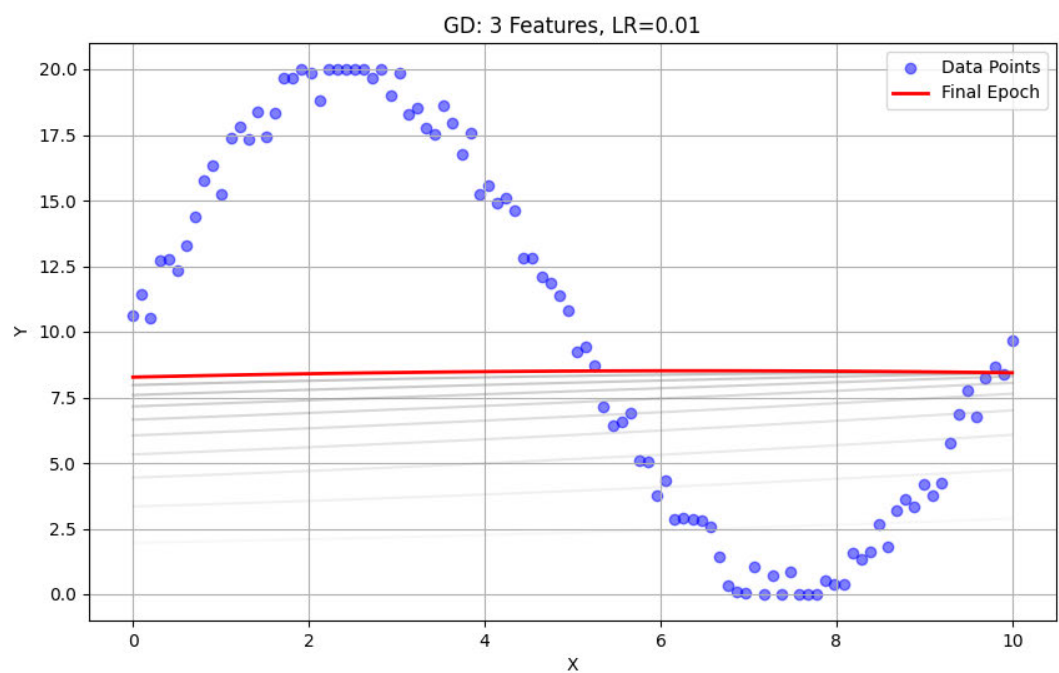


Figure 3a: Standard Gradient Descent showing smooth, calculated, but slow updates over 100 epochs.

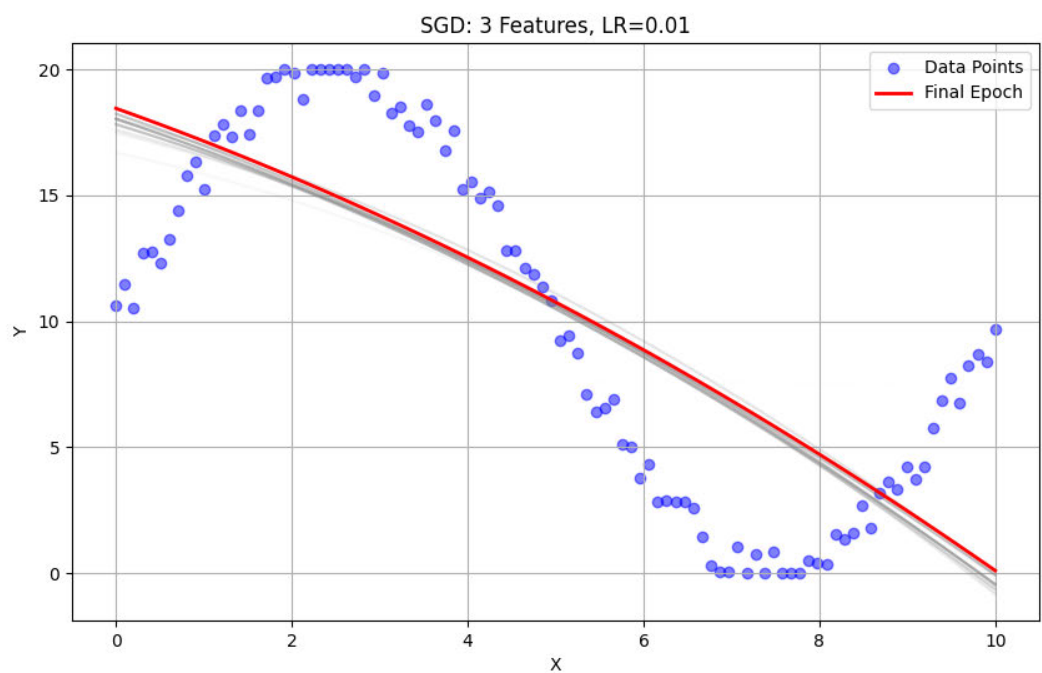


Figure 3b: Stochastic Gradient Descent showing rapid, slightly noisier convergence to a similar line in the same number of epochs.

The Impact of Learning Rates

At a learning rate of 0.01, the model took the full 100 epochs to gently approach the optimal curve. By increasing the learning rate to 0.10, the model learned significantly faster, taking larger steps toward the minimum loss. Because our data was strictly normalized, the higher learning rate did not cause the model to diverge.

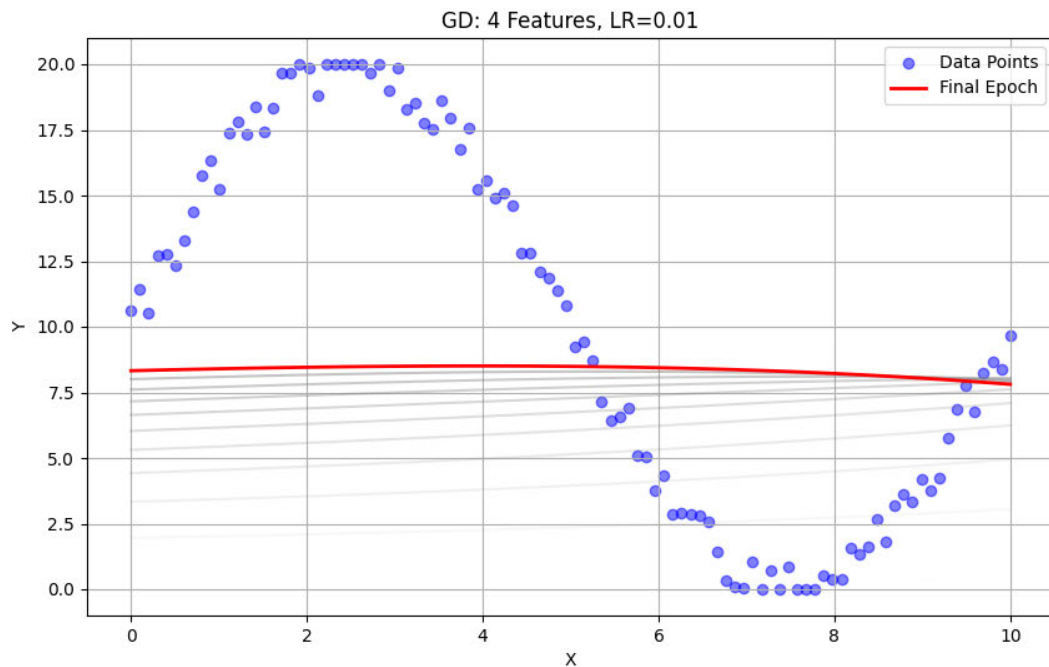


Figure 4a: With a low learning rate of 0.01, the model learns slowly and the epoch updates are tightly packed together.

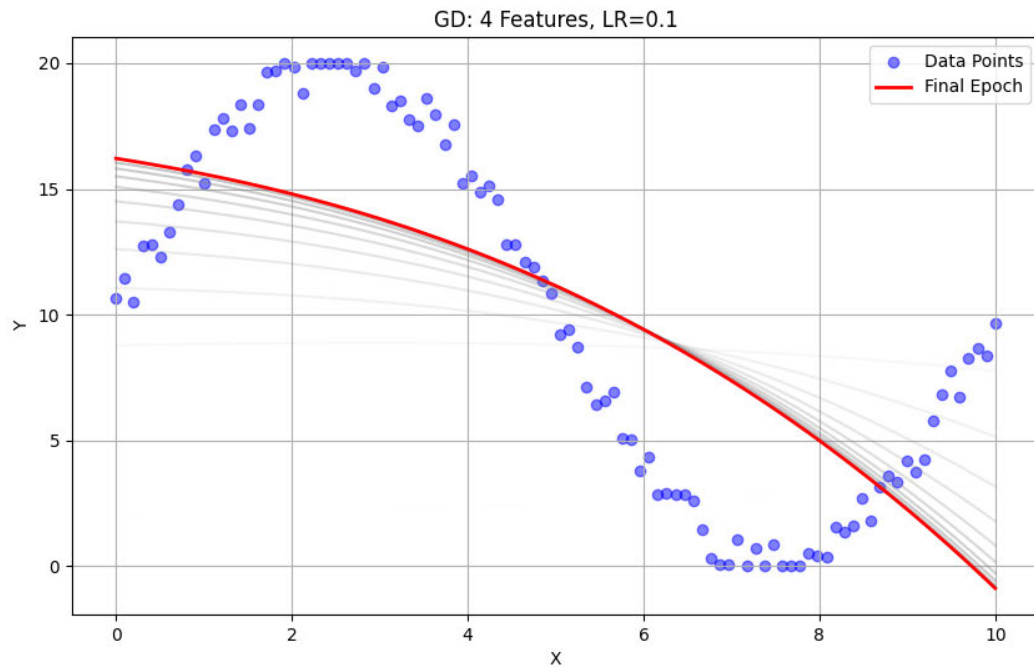


Figure 4b: With a high learning rate of 0.1, the model takes larger steps, spreading the epoch lines further apart as it rapidly seeks the minimum.

Part 2: Neural Network Application - Fashion Image Classification

1. Motivation and Significance

For the second phase of this project, our team chose to tackle a problem in the domain of Computer Vision: automating the categorization of clothing items. In the modern e-commerce and retail logistics sectors, millions of images must be tagged and sorted daily. Automating this process using deep learning drastically reduces operational costs, improves search accuracy for consumers, and streamlines inventory management.

2. Methodology

To solve this problem, we utilized the Fashion-MNIST dataset via TensorFlow/Keras. This dataset consists of 60,000 training images and 10,000 testing images of 28x28 grayscale clothing items, categorized into 10 distinct classes (e.g., shirts, sneakers, bags).

We constructed a Feed-Forward Neural Network with the following architecture:

- Input Layer: A Flatten layer to convert the 2D 28x28 images into a 1D array of 784 pixels.

- Hidden Layer: A Dense layer consisting of 128 neurons utilizing the Rectified Linear Unit (ReLU) activation function to introduce non-linearity.
- Output Layer: A Dense layer with 10 neurons utilizing a Softmax activation function to output the probability distribution across the 10 clothing classes.

To tie this application back to the core themes of the class, we compiled the model using the Stochastic Gradient Descent (SGD) optimizer and sparse categorical cross-entropy as our loss function.

3. Results

The model was trained over 15 epochs. It successfully learned to map the pixel intensities to their corresponding categories.

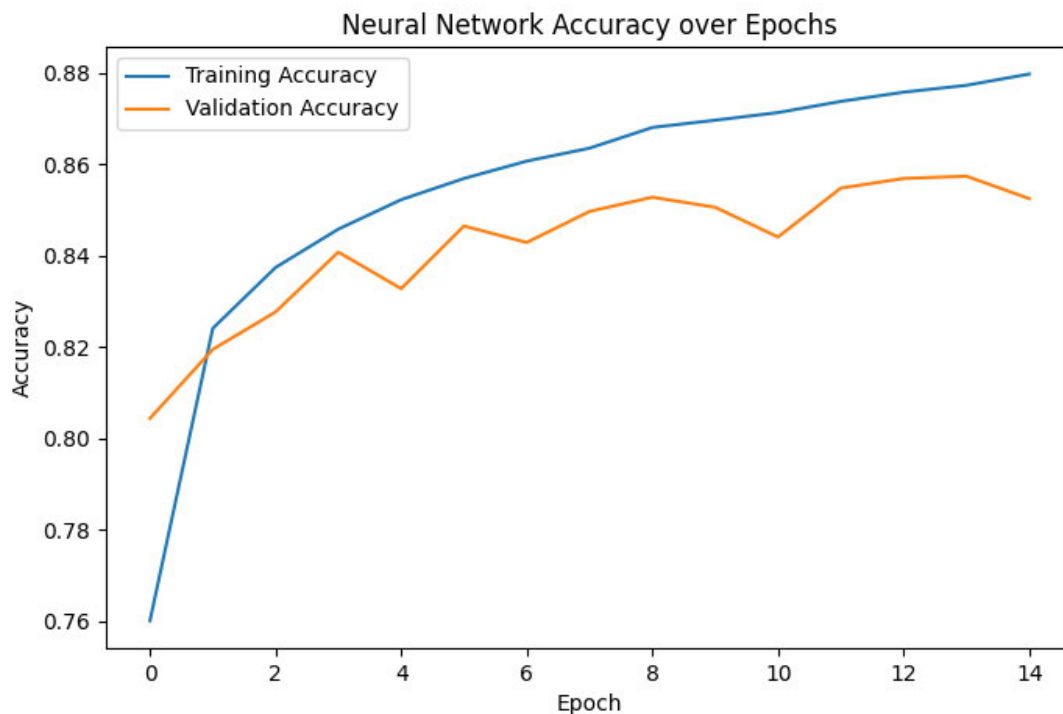


Figure 5: Training and Validation Accuracy over 15 epochs.

The neural network achieved a highly respectable Test Accuracy of 85.25%. The training curve indicates steady learning, with the validation accuracy closely mirroring the training accuracy, confirming that our model generalized well to unseen data and did not suffer from severe overfitting.

4. Strengths and Limitations

Strengths:

- Efficiency: The lightweight architecture of the feed-forward network allowed for extremely rapid training times while still achieving high accuracy.
- Simplicity: The model is highly interpretable and requires minimal hyperparameter tuning to yield effective baseline results.

Limitations:

- Spatial Ignorance: By using a Flatten layer, the network destroys the 2D spatial structure of the image. It evaluates pixels independently rather than recognizing local patterns like edges, curves, or textures (e.g., the strap of a bag or the sleeve of a shirt).
- Future Work: To improve accuracy beyond the ~85% threshold, the methodology should be upgraded to a Convolutional Neural Network (CNN), which uses filter kernels to preserve and analyze the geometric features of the images.

5. References

- Xiao, H., Rasul, K., & Vollgraf, R. (2017). Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. *arXiv preprint arXiv:1708.07747*.